DevWell

Member 1 | Team handoff

Authentication, onboarding and backend integration

ITDMA3-34 - Team 3 - 23 September 2026

Owner: Themba / Member 1. Course: ITDMA3-34, Team 3.

What changed

Real email/password sign-in and engineer registration replace role-based login. The account profile supplies the role and approval state. New engineers choose an active company and matching team, submit a join request, and wait for approval. Pending/rejected accounts do not get a protected navigation graph. The graph is recreated for each signed-in user and role, so sign-out removes previous navigation. Passwords are not put into saved UI state. Supabase sessions use device-key encryption; Supabase handles refresh tokens. Application backup is disabled to avoid restoring tokens.

The SDK versions are pinned: Supabase Kotlin 3.0.3, Ktor 3.0.2, Kotlin/Compose compiler 2.1.0. These match the existing Android Gradle Plugin 8.7.3 and SDK 35 project.

Run in Android Studio

1. Open this repository root (the folder containing settings.gradle.kts).
2. Select JDK 17 for Gradle and install Android SDK 35 when prompted.
3. Add SUPABASE_URL and SUPABASE_ANON_KEY to local.properties. Use a publishable key, never a service-role/secret key. This file is excluded from Git.
4. Sync Gradle, choose the app debug configuration and an API 26+ emulator/device.
5. Run the app. Sign in once the backend below is ready.
6. While the backend is being built, expand Student preview (debug only). Preview an existing role or a new engineer. The pending preview can simulate approval and rejection. These actions use memory only. Restarting the process ends preview.
7. Release builds hide preview and also reject preview actions in the ViewModel.

The project URL is https://whtsjwiqcesngltfghyb.supabase.co. Obtain the public key from the project Connect/API Keys panel. The key is embedded in the installed app; row-level security, not hiding a publishable key, protects the database.

Simple code map

- ui/auth: form fields, loading, errors, company/team selection, approval screens.
- viewmodel/AuthViewModel.kt: actions and UI state; blocks repeated taps.
- data/auth/SupabaseIdentityRepository.kt: real account and onboarding requests.
- data/auth/IdentityRows.kt: JSON column names mapped to the team's domain models.
- data/auth/DeviceSessionManager.kt: saves SDK sessions encrypted on the phone.
- data/auth/PreviewIdentityRepository.kt: separate in-memory student preview.

- model/EntryDestination.kt: readable role/status rules and form validation.
- ui/navigation/DevWellApp.kt: auth gate and role-specific navigation graphs.

Screens call ViewModel functions; ViewModels call repositories. Other feature repositories still use the starter's sample data and must be connected by their owners. The app labels those dashboards as sample data. Do not treat preview approval as a real manager approval, or mocked manager reviews as database updates.

Database contract (backend owner)

No schema or Edge Function changes were applied by this app task. At the initial inspection on 23 September 2026 the public schema contained no tables. The following is the contract the Android implementation expects; coordinate names before changing it. All IDs below are UUIDs in production, represented as Kotlin strings.

public.user

- id: UUID primary key, references auth.users.id (the same ID; not a new generated UUID).
- name: non-null text; email: non-null text from the authenticated account.
- role: non-null text, one of engineer, manager, admin.
- status: non-null text, one of pending, active, rejected.
- company_id: nullable company foreign key; team_id: nullable team foreign key.
- onboarding_done: non-null boolean.
- New signup: role=engineer, status=pending, onboarding_done=false, no membership.
- Submitted request: status=pending, onboarding_done=true, selected membership IDs.
- Approved: status=active, onboarding_done=true, correct company and team.
- Rejected: status=rejected; keep onboarding_done=true so the account stays blocked.
- Provision manager/admin accounts through trusted backend/admin operations only.

Create each profile from trusted backend logic after Auth registration. A display name may come from user_metadata.name, but role/status/membership must never be trusted from user-editable metadata. The app refuses to enter a dashboard without a readable profile.

public.company

- id: UUID primary key; name: non-null text.
- status: pending or active. Only active rows are selectable.
- email_domain: optional nullable text (not used to grant membership).

public.team

- id: UUID primary key; company_id: company foreign key; name: non-null text.
- manager_id: optional user foreign key for manager authorization.
- Other members may add cadence/timezone/threshold fields. Member 1 reads only id, company_id and name. Return an empty list when no teams are available.

public.join_request

- id: UUID primary key; user_id: user foreign key; company_id and team_id: foreign keys.
- status: pending, approved or rejected; created_at: timestamp with timezone.
- reviewed_by and reviewed_at: nullable reviewer ID and timestamp.

- Prevent duplicate pending requests for the same engineer with a database constraint.
- Index user_id, team_id and company_id when used by filters/access policies.
- Enforce that the selected team belongs to the selected company.

Edge Function: submit-join-request (required by Member 1)

Method: POST /functions/v1/submit-join-request. Headers: apikey (public key), Authorization: Bearer <signed-in user's access token>, Content-Type: application/json. The Supabase Kotlin SDK attaches these.

Request body contains only company_id and team_id:

```
{"company_id":"<company UUID>","team_id":"<team UUID>"}
```

Validate the access token with Supabase Auth. Derive the user ID from that verified identity. Check the profile is an engineer eligible for onboarding, the company is active, and the team belongs to it. Never accept user_id, role or status from the app.

Atomically insert the pending join_request and set the profile's company_id, team_id, status=pending and onboarding_done=true. Use a transaction through a carefully secured database function; two unrelated REST writes are not atomic. If a privileged helper is needed, keep it outside exposed schemas, restrict execute grants, fix search_path, and validate the caller/ownership. Do not grant users direct role/status updates.

Make retries idempotent: an identical pending request returns success; a conflicting request returns a conflict. A successful HTTP response must mean both writes committed. Suggested success: HTTP 200/201 with {"status":"pending"}. The current Android app uses HTTP success then re-reads public.user; it does not require a particular JSON response body. Never return 200 for an error. Use 400 for invalid input, 401 for an invalid/missing session, 403 for forbidden actions, 409 for conflicts, 500 for failure.

Keep JWT verification enabled and validate identity inside the function. Keep server keys in server secrets only. Do not log access tokens, passwords or wellbeing data.

Approval contract (Member 5 + backend owner)

Member 1 refreshes public.user when Refresh status is tapped. No Realtime subscription is required for this version. The manager's backend operation must verify the reviewer manages the requested team and atomically update the request and user profile. Approve sets request.status=approved and user.status=active; reject sets both to rejected. Always retain onboarding_done=true and the correct membership IDs. The starter's MockManagerRepository does not perform this live operation.

A suggested review-join-request Edge Function may accept request_id and approve; this is a recommendation for Member 5, not an endpoint currently called by Member 1.

Access rules and integration notes for the whole team

- Enable RLS on every exposed table and add only necessary Data API grants. A table grant alone is not row authorization. Test policies with ordinary signed-in users.
- Engineers read their own profile. Prevent self-editing role/status/membership.
- Signed-in onboarding users can read active companies and their teams. Expose only the directory fields needed for selection; the UI is not an access-control boundary.
- Engineers can read only their own join requests. Managers can access requests only for teams they manage. Do not grant manager access to individual wellbeing records.
- Pending/rejected accounts must be blocked by backend policies for all protected check-ins, projects and other features, even if someone bypasses the Android UI.

- Member 2: write check-ins for auth.uid(), after active membership checks.
- Member 3: personal history is own-user data; privacy preview uses the same safe aggregate as Member 5. The existing minimum aggregate response count is 5.
- Member 4: projects/tasks require user/team authorization; never accept ownership from untrusted request fields without checking it.
- Member 5: live request review must update the profile as described above.
- Member 6: notifications, AI and admin remain your features. AI server secrets and privileged company/role changes belong on the backend. Sign-out is already shared.

End-to-end acceptance checklist after backend deployment

1. Register an engineer, confirm the email, sign in, and read the new profile.
2. Show only active companies; changing company clears the selected team.
3. Submit once, then retry after a timeout: exactly one pending request remains.
4. Pending account cannot open protected screens or access protected REST data.
5. A manager for a different team cannot approve this request.
6. Correct manager approves; engineer taps Refresh status and reaches Home.
7. Rejected account shows a clear blocked screen and cannot enter protected features.
8. Manager/admin accounts route to their own dashboards based on trusted profiles.
9. Restart the app and verify saved-session restore; revoke a session and test expiry.
10. Sign out online and offline; restart/Back must not reopen the previous session.
11. Check weak password, incorrect password, unconfirmed email, empty directory, network loss, device rotation, and double taps. No app crashes or stuck spinners.
12. Build release and confirm no student preview can be entered.

Email confirmation currently happens in the email/browser; return to the app and sign in manually. Configure a usable confirmation landing URL in Supabase Auth. Password reset, Google sign-in and optional manager company registration are not part of this change.

References

- Supabase Kotlin installation: https://supabase.com/docs/reference/kotlin/installing
- Pinned SDK source/API: https://github.com/supabase-community/supabase-kt/tree/3.0.3
- Row-level security: https://supabase.com/docs/guides/database/postgres/row-level-security
- Edge Function authentication: https://supabase.com/docs/guides/functions/auth

See validation.md for the actual checks performed. Backend-dependent checks above are not marked passed until the database and functions are deployed and tested together.

Verification record

Date: 23 September 2026.

Environment

Windows, JDK 17, Android SDK 35, Android Gradle Plugin 8.7.3, Gradle wrapper 8.14.3 and Kotlin 2.1.0. Device checks used the DevWellTest Android 16 / API 36 x86_64 emulator at 720 x 1600.

Verified checks

- Debug APK built, installed and launched on the emulator.
- Unsigned release APK built successfully; release vital lint also passed. A Windows Gradle cache move failed on the first attempt; a single-worker retry passed.
- All 23 JVM unit tests passed: role/status routing, validation, repeated taps, session loss, repository responses, offline sign-out and existing privacy/tips rules.
- All 7 emulator tests passed: validation, onboarding selection and approval gates, manager/admin navigation and sign-out, rotation, encrypted session round-trip and corruption.
- Android lint completed with 0 errors and 11 warnings. Warnings cover dependency/SDK updates, the starter's missing launcher icon, backup configuration, and a synchronous preference write deliberately performed on the IO dispatcher.

Scope and remaining integration

The tests use controlled responses or the isolated student preview. The device session tests use Android's real encrypted storage. Live registration, email confirmation, join-request submission, RLS and manager approval must still be tested once the backend owner deploys the tables and Edge Functions described in the handoff contract. No database tables or Edge Functions were created or changed by this app task.

Other members' dashboards still use sample repositories. This is Member 1's app-side implementation, not a claim that the whole team application is connected to live data. Android Studio's underlying Gradle build and an Android emulator were exercised; opening and syncing through the Android Studio editor UI was not automated.

Local Windows build note

This machine required JAVA_TOOL_OPTIONS=-Djdk.net.unixdomain.tmpdir=C:/devwell-unused-socket-directory for Gradle's local communication startup. The path must remain nonexistent to use the TCP fallback. This is a process environment workaround for this machine, not an app setting. If the same startup error occurs in Android Studio, set that environment variable before launching the editor. Normal installations may not need it.

Use JDK 17 and the checked-in Gradle wrapper. The release APK is unsigned; use the debug APK for local installation or configure your own release signing outside Git.